

The polymorphic menagerie

Class hierarchies, relationally caged

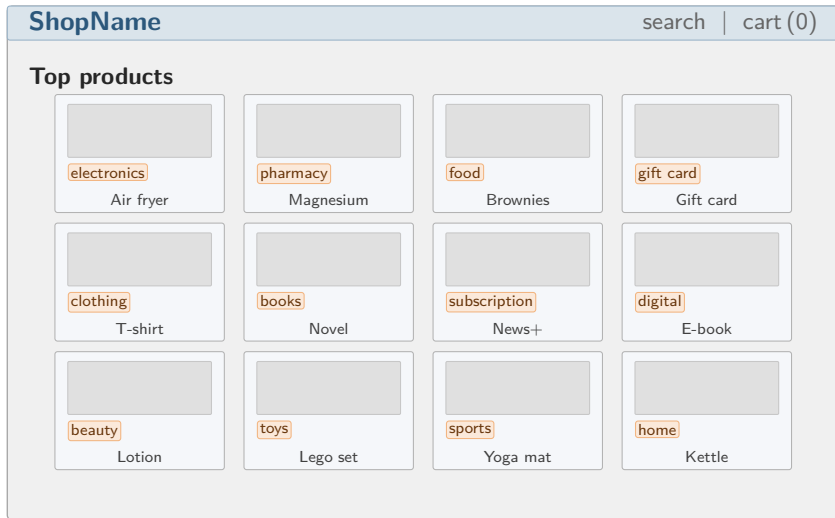
Andrei Lepikhov

pgEdge

2026



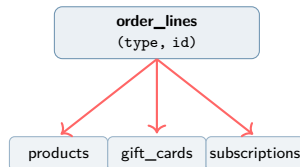
A page you've already seen



The menagerie's three habitats.

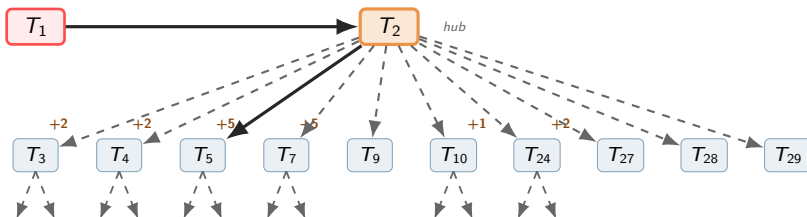
- **Catalogs** — e-commerce front pages, 1C Trade, ERP item registers
- **Activity / audit** — Salesforce WhoId/WhatId, Jira, audit logs
- **Documents / workflow** — SAP, Hibernate @Inheritance(JOINED), BPM tasks

Generated reflexively by ORMs (Rails, Django, Hibernate) and deliberately by platforms (Salesforce, 1C). Same query shape, same bottleneck.



No FK spans multiple targets
⇒ encoding workaround

What 1C actually generates



Legend:

- $\longrightarrow$ INNER JOIN (2 of them)
- $--\rightarrow$ LEFT OUTER JOIN (26 of them)

305 lines · 29 aliases · 28 joins · 0 EXISTS

(+N = grandchildren hidden; actual table types repeat — e.g. `_Reference189` appears 5 times)

The Query

```
SELECT
  ol.id,
  COALESCE(p.name, g.name, s.name) AS item_name
FROM order_lines ol
  LEFT JOIN products p
    ON ol.type = 'product' AND ol.id = p.id
  LEFT JOIN gift_cards g
    ON ol.type = 'gift_card' AND ol.id = g.id
  LEFT JOIN subscriptions s
    ON ol.type = 'subscription' AND ol.id = s.id
WHERE EXISTS (
  SELECT 1 FROM orders o
  WHERE o.id = ol.id AND o.placed_at >= 2024/01/01)
ORDER BY ol.popularity
LIMIT 100
```

*12 categories on the front page $\Rightarrow$ 12 candidate target tables in the schema;
we trace 3 here.*

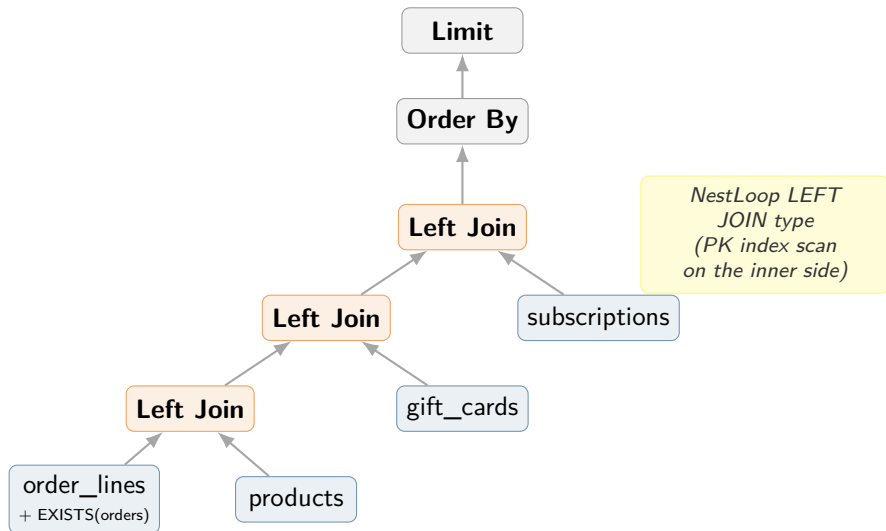
Why it blows up: the per-row mismatch

The discriminator picks one target per row.
The planner picks one probe order for *all* rows.

	products	gift_cards	subscriptions	
type='product'	✓	×	×	
type='gift_card'	×	✓	×	useful: $1/N$
type='subscription'	×	×	✓	

wasted: $(N-1)/N$ per row

Query Plan Tree



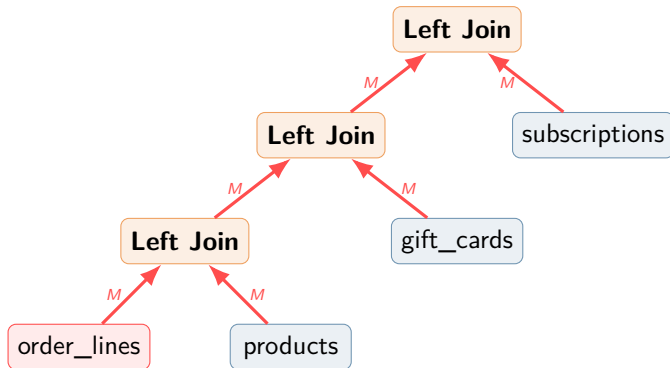
Main Execution Cost Amplifiers

- Base table (`order_lines`) growth
- Growth in the number of LEFT JOINs
- Ordering column of the base table not covered by an index
- EXISTS pull-up displaces the filter outside the optimisable block

Base Table Growth

*When M doubles, every probe count
doubles with it*

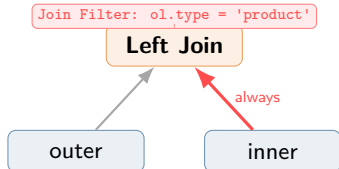
Amplifier: Base Table Growth



Every edge carries M rows. Doubling M doubles the number of probes on the inner side.

Concept: Result Filter

Current: probe, then filter



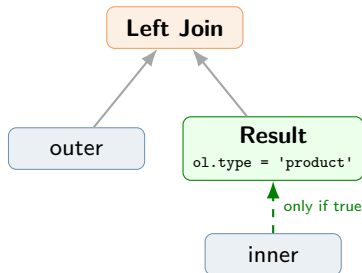
Inner scan runs **unconditionally**.

Filter rejects result *after* the probe.



Thread on [pgsql-hackers](#)

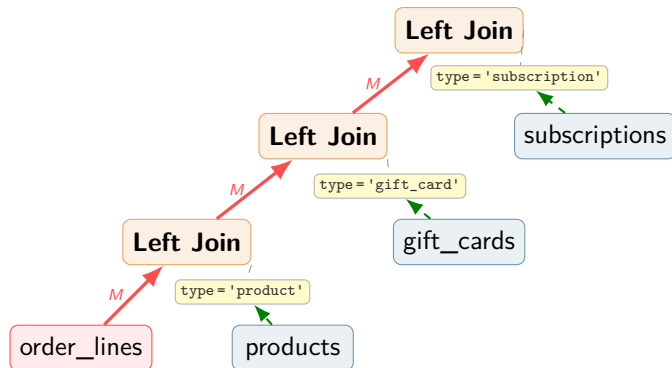
Proposed: Result filter atop RHS



Result node evaluates LHS-only clause as a **one-time filter**.

If false → returns $\emptyset$; inner scan *never launched*.

One-Sided Clause Guard



Guard checks discriminator *before* probing inner side. Each row probes only the matching table: $M_1 + M_2 + M_3 = M$.

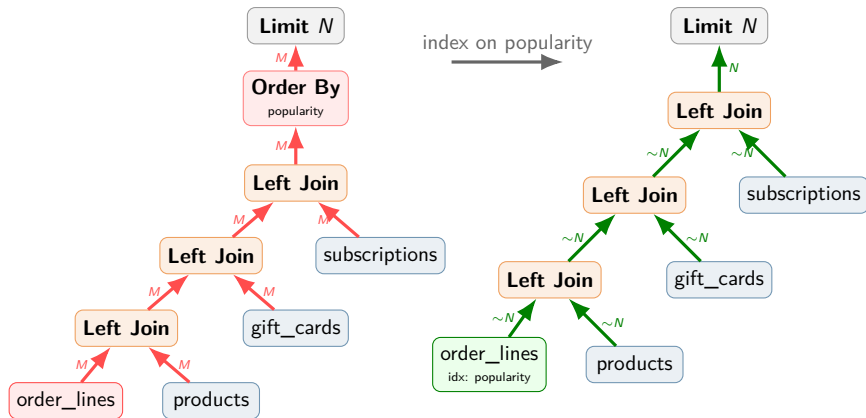
Benefits of the Result Filter Approach

- **Fewer page accesses.** Each base row probes only the *matching* inner table instead of all N . Total index/heap page reads drop from $M \times N$ to M .
- **Safety net for missing indexes.** When no index covers the inner side, the executor would fall back to a sequential scan *per outer row*. The guard lets it skip the scan entirely for non-matching rows.
- **Limiting price of a new LEFT JOIN.** Each new category in the model means one more LEFT JOIN at each query. With a guard clause, such new JOIN operator adds to the executor only strictly necessary actions without recursion into the inner subtree.

Aggregated Result

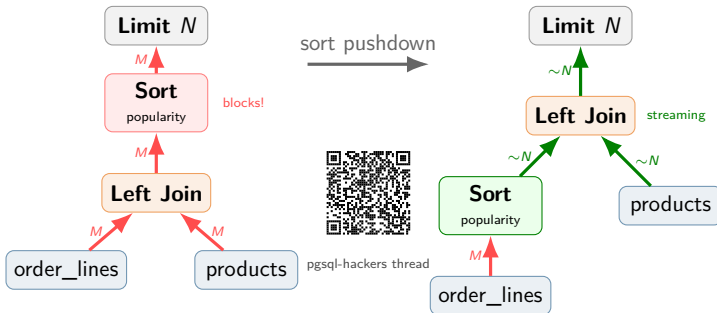
*ORDER BY + LIMIT,
and the blocking Sort that gets in their way*

ORDER BY + LIMIT: Effect of Proper Index

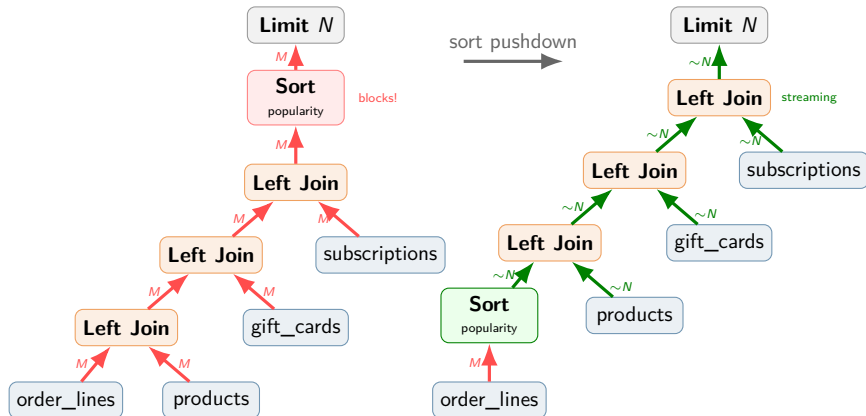


$N = LIMIT$. Index-ordered scan eliminates the blocking **Order By**. **Limit** stops the entire **LEFT JOIN** chain after N rows: $M \times 3 \rightarrow \sim N \times 3$ probes.

What if no proper index exists?



Sort Pushdown: Presorted Outer Path

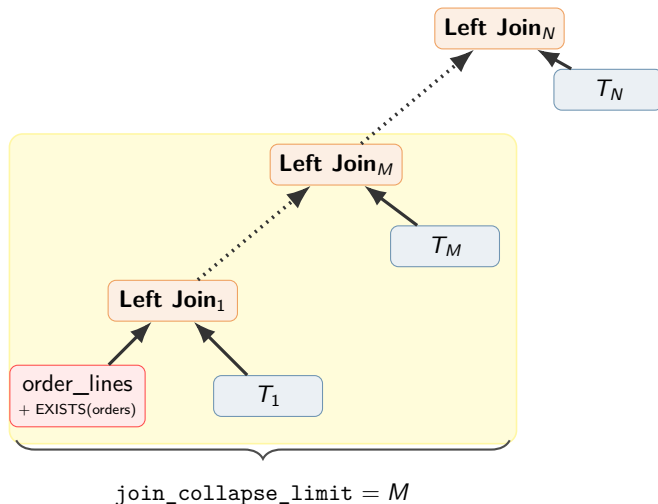


Push Sort below the joins: base rows sorted once ($M \log M$), but only $\sim N$ enter the join chain. Probes: $M \times 3 \rightarrow \sim N \times 3$.

The SubLink pull-up problem

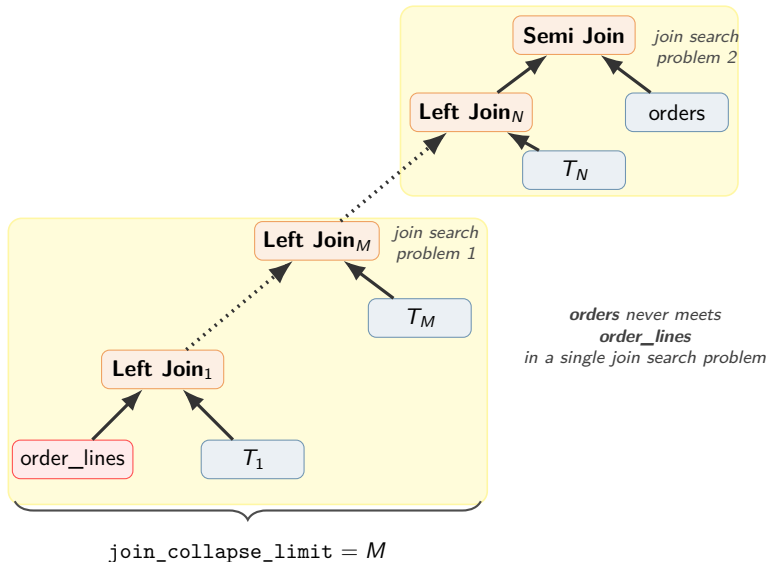
*EXISTS, the join_collapse_limit cliff,
and what happens between them*

An N -deep query tree

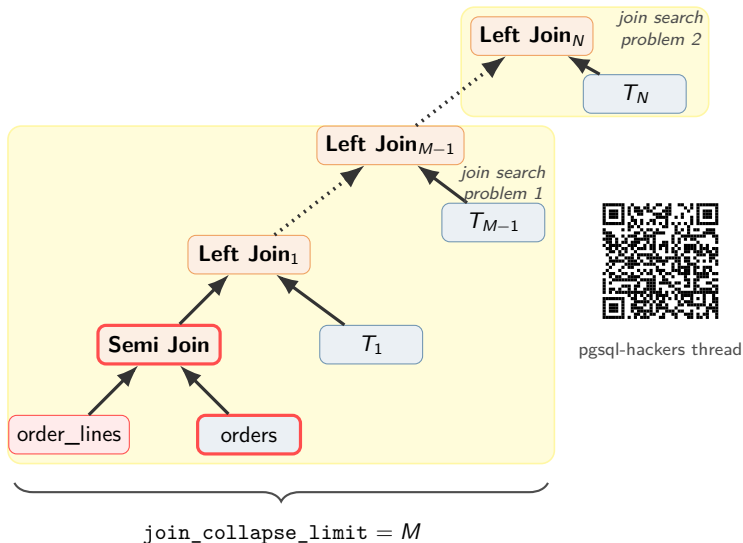


One problem — EXISTS lives on order_lines, inside the M-block.

An N -deep query tree after EXISTS pull-up



An N -deep query tree after SubLink relocation



How many rows, really?

*An adjacent problem —
join cardinality across N joins*

The estimates the planner trusts

PostgreSQL costs each join under the **independence assumption**:
 $|A \bowtie B| \approx |A| \cdot |B| \cdot \text{sel}(A.x = B.y)$, where the selectivity is read from single-column statistics on each side.

- Leis et al. (VLDB 2015) measured these estimates on the JOB benchmark:
errors compound — the median 4-way join estimate is off by $10\times$, the worst by $\geq 1000\times$.
- For our polymorphic shape this hits twice over:
 - the join column distribution is *conditional* on the discriminator;
 - the per-row mismatch (slide 6) means only one branch contributes, but the planner does not see that until execution.
- Consequence: even when Result Filter, Sort Pushdown, and SubLink relocation produce the right *shape*, the planner may still pick the wrong join order because the *cost* of that shape is wrong.

The planner is solving the right problem with the wrong numbers.

Optimizer statistics for joins

The proposal (Huinker, Wang; pgsql-hackers, Dec 2025–): collect statistics *about the join itself* — not just about each side independently.

For a frequently joined pair ($A.x, B.y$), store:

- the matching row count,
- the join-key distribution in the result,
- optionally, per-discriminator breakdowns.

Where it comes from:

- declared FKs → exact upper bound, for free;
- ANALYZE can sample joins like it samples tables;
- extended statistics is the natural plug-in point.

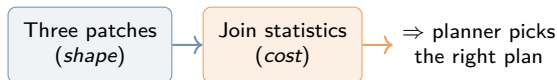


pgsql-hackers thread
*“Is there value in having
optimizer stats for
joins/foreignkeys?”*

C. Huinker, A. Wang, et al.
Dec 2025–

Why this matters for polymorphic shapes

- **Result Filter** is only chosen if the planner believes the filter is selective. Today the selectivity of `o1.type = 'product'` on the joined side is a guess; join-level stats turn it into a measurement.
- **Sort Pushdown** is only chosen if the planner believes the presorted path is cheap enough. Join-level stats give a real row-count for the sort input, instead of the product of two unconditional marginals.
- **SubLink relocation** reshapes the tree across $\geq M$ joins. The whole point is that the planner gets to compare alternatives — it can only do that if the alternatives are costed correctly.



The three patches and join statistics are complementary: shape without cost is a lottery; cost without shape is precision aimed at the wrong target.

Q & A

Patches on postgresql-hackers



One-sided clause guard
(Dec 2024)



Presorted outer path
(Apr 2026)



SubLink relocation
(May 2026)

In the literature

- M. Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002
- B. Karwin, *SQL Antipatterns*, Pragmatic Bookshelf, 2010 — ch. “Polymorphic Associations”
- V. Leis et al., *How Good Are Query Optimizers, Really?*, VLDB 2015
- Rails Guides, Salesforce SOQL TYPEOF, GitLab handbook (polymorphic associations banned), django-polymorphic